

Scotland Yard Report

Archie Kind , Tom Lam
21/3/2025

Model

We have completed the closed part (cw-model) of the assignment and passed all 83 tests.

AI

Move filtering

We used the following filters for each model.

Model	Filters
All	- Only allow secret and 1 other type of ticket to every destination - Only allow secret tickets when Mr X is not in the first 2 rounds, not a reveal move and not on a location with only taxi station - Only allow double moves if all Mr X moves will lead to capture in the next round
Paranoid	Allow Mr X to use up remaining secret tickets at the end of the games if there are any left
Paranoid & MCTS	Eliminate the move order of the detectives*

Fig 1: Move filters used for each AI model

* In minimax and 1 step look ahead this is done by returning only 1 piece's moves based on the minimum distance to a determinised location of Mr X. In MCTS we select the piece to move that minimises the total distance to all Mr X possible locations for the detectives but use the same filters as minimax for Mr X. This gives Mr X a pessimistic estimate of move order.

Sample Mr X possible locations

We maintain a set of the possible locations of Mr X. When Mr X moves, the locations are recalculated based on the tickets used. When a detective moves, the destination is removed from the set of possible locations.

1. One Step Look Ahead

1.1. Mr X

$$m_{best} = \arg \max_{m \in M} \min \left(D_{shortest}(G'(m)) \right)$$

Mr X will always pick the move m_{best} that maximize the distance to the closest detective, where M is the set of filtered moves, $G'(m)$ represents the new game state after advancing with the move m , and $\min \left(D_{shortest}(G'(m)) \right)$ is the distance from Mr X to the closest detective after move m computed with Dijkstra's shortest path algorithm.

1.2. Detectives

$$m_{best} = \arg \min_{m \in M} \sum_{l \in L} dist(dest(m), l)$$

The detectives pick the move m_{best} that minimize the total distance to Mr X, i.e. $\sum_{l \in L} dist(dest(m), l)$ for each location l in the set of all Mr X's possible locations L .

2. Paranoid Minimax

The general structure of the search tree consists of repeated levels of MAX nodes (Mr X) and multiple MIN node (Detective). All detectives are trying to minimize the score of Mr X and while Mr X does the opposite. In our current version of the minimax, we only allow for 1 level for every player including the detectives, which there will be a total of 6 levels.

In the terminal state of the search, we used the following evaluation function modified from [1]

$$score_{MrX} = 100 \times \min_{i \in D} (d_{detective, i}) + 10 \times t_{MrX, BF} + |L| + \rho$$

where $d_{detective}$ is the distance from the location of each detective to Mr X and $t_{MrX, BF}$ is the number of black tickets Mr X have at the current game state. I have also added a variable L , the size of Mr X possible locations to ensure that Mr X will prioritized moves that maximize the uncertainty of the detectives and multiply the $t_{MrX, BF}$ by a factor of 10 to preserve secret tickets.

To further increase the performance of the minimax tree search, we have utilized a simple alpha-beta pruning mechanism to eliminate branches that are guaranteed to produce a worse outcome than the previously evaluated nodes.

3. Expectimax Minimax

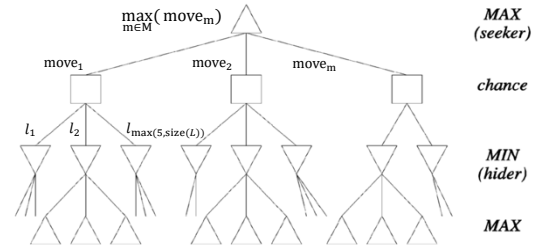


Fig 2: Structure of the expectimax tree for the seekers (modified from figure 6 in [1])

Our expectimax minimax is a variant of minimax where we expand every possible move m of the detective at the root level with a subtree of every possible Mr X move. Our version simplifies the original expectimax algorithm mentioned in [1], as we populate the root of each subtree without assigning separate weights, meaning that no Mr X location l is prioritized or discouraged. Our search tree consists of 5 levels, which includes 3 detective levels, 2 Mr X levels and we only randomly sample 5 Mr X possible locations for the every subtree for each move.

$$m_{best} = \arg \max_{m \in M} \sum_{l \in L} minimax_{detective, MrX}(G_l, d, MrX)$$

The algorithm will pick the move with corresponding to the subtree with the highest total score, i.e.

$\sum_{l \in L} \text{minimax}(G_l, d, \text{MrX})$, where G_l is the game state with MrX 's location set to one of its possible location l .

In order to allow a deep search in the game tree, we only include a randomly selected detective and Mr X in a given game state, ignoring other detectives' actions.

A large modification we made compared to the original paper [1] is the update of Mr X's possible locations. We kept the set constant during in recursion and only updated it at the start of each move selection function call. This change significantly reduces the size of the set in the evaluation function which has shown to produce a higher win rate of the detectives. A possible explanation is that the more moves added to the set, the more the noisy the evaluation is which might cause the detectives to make a move away from Mr X. While it is not the most accurate to assume the location Mr X does not change throughout the recursion, it ensures the detectives will most likely move around the previous revealed locations of Mr X.

4. Monte Carlo Tree Search

4.1. Selection

Node selection occurs by applying the UTC algorithm to each layer of the game tree.

$$UTC = \frac{r_i}{n_i} + C \sqrt{\left(\frac{\ln(n_p)}{n_i}\right)} + W \frac{\left(\frac{r_a}{n_a}\right)}{n_i \left(1 - \frac{r_i}{n_i}\right) + 1}$$

Here r_i is the total reward at node i , n_i is the total number of visits and n_p is the total number of visits to the parent node. The progressive history was added as suggested by [1]. The values of the constants at $C = 0.5$ and $W = 5$ were also taken from the experiments in [1].

4.2. Expansion

A node for each successor state of node in question is added. A key efficiency gain (particularly for the detectives) was to instantiate child nodes without immediately instantiating their game state having calculated available moves from the parent node. The game state is only then instantiated if necessary.

4.3. Payout

Mr X and the detectives all use payout strategies suggested in [1]. Mr X selects the move that maximises the distance to the closest detective. The detectives pick the moves that minimise the total distance to all possible Mr X locations. Throughout Mr X and the detectives make random moves with probability 0.1 and 0.2 respectively.

4.4. Backpropagation

The reward is backpropagated up the tree (1 if Mr X wins the game, 0 if he loses). The detective's reward is discussed in 4.5.

4.5. Detective Move Selection and Coalition Reduction

[1] Observed the problem that detectives, when they are far away from the approximate location of Mr X, can sometimes rely on the other detectives to win and so do not make reasonable moves. The paper proposed "coalition reduction" to mitigate this. A detective's reward would be 1 if it was that specific detective who captured Mr X in the simulation, and it would be $(1 - r)$ otherwise. We solved this issue using this strategy in combination with a term encouraging moves closer to Mr X

$$\text{move} = \max \left\{ \frac{x_i}{n_i} - c \times \min\{d_k\} \right\}$$

Here x_i is the average reward of the move, n_i is the number of visits, d_k is the distance of the detective to the closest possible location of Mr X and c is a very small constant (we chose 0.02). This allows movement towards Mr X to be prioritised when the detective is far away and there is little spread in the reward, but the term is insignificant when the detective is closer and the variance in the reward is greater. The technique is successful in reducing Mr X's win rate from 80% to 55% (coalition reduction used in both cases).

4.6. Single Tree Determinisation

One of the key attributes of Scotland Yard is the fact that it is an asymmetric game with hidden information. [1] proposes both single and multi-tree determinisation in order to deal with the hidden information when selecting a detective move. In the latter, a separate tree is generated for each possible location of Mr X. The move chosen based off combination these data. Single tree determinisation generates only one tree, but with all moves from all possible locations generated from the first hider ply. This allows the attributes of each node to be updated based on every payout rather than a subset of them.

Determinisation Method	Mr X Win Rate
Detective Single-Tree with Root Parallelisation	30%
Detective Single-Tree without Root Parallelisation	45%
Detective Multi-Tree in Parallel	50%

Fig 3: 40 iterations vs Mr X with root parallelisation

4.7. Root Parallelisation

We build multiple (16) MCTS trees in parallel. We then combine the returns from each child node or the root node of each tree and then select the 'best' child node. [2] showed that this method is superior to other parallelisation methods such as leaf parallelisation (where the payouts are done in parallel) and tree parallelisation (where the building of an individual tree is done in parallel).

Parallelization method	Mr X win rate
Mr X Root Parallelisation	55%
Mr X Sequential	50%
Detective Root Parallelisation + Single Tree Determinisation	55%
Detective Sequential + Single Tree Determinisation	100% (10 iterations)

Fig 4: Mr X statistics collected versus Detective Root Parallelisation + Single Tree Determinisation. Detective Statistics collected versus Mr X Root Parallelisation.

Whilst it appears to have little effect on Mr X's win-rate, it is clearly an essential technique for the detectives. This could be because they have a larger state space due to the hidden information of Mr X's possible locations that benefits to a greater extent from additional exploration. The constants chosen for UCT already encourage sufficient exploration for Mr X.

Design Patterns

5.1. Factory Pattern

In the dropdown menu of the GUI, we can choose between One Step Look Ahead, Minimax and MCTS. To determine whether the Mr X or detective's model to be activated, we created 3 model factories which produce the 3 sets of AI pairs. In the AI child classes, we will call pick move on the corresponding model based on who commences the move.

5.2. Singleton Pattern

In order to compute the shortest path from Mr X to the detectives, we created a Dijkstra's class that pre-compute the distance between every nodes in the graph on initialization. In order to prevent creating additional instance, we made it a singleton which only allows one Dijkstra object in memory.

Unit Tests

6.1. Heuristics Tests

We created 2 test classes to test the Dijkstra and the MrXPossibleLocations class. We tested the shortest path methods, the update of Mr X's locations, with 2 test cases for each method.

6.1. AI Model Tests

We tested the one step look ahead model for the detectives and Mr X each with 2 test cases for pickMove. For the MCTS algorithm, we tested the UTC selection strategy, node instantiation, expand and select best child, which are the helper methods of both MCTS models.

Experiments

7.1. CPU and Memory Usage

Model	Multi-Threaded?	Max CPU	Max Memory
OneStep	No	11%	357MB
Minimax	Yes	40%	863MB
MCTS	Yes	84%	1682MB

Fig 5: The CPU and memory usage of each AI model. Run on an 12 Generation Intel Core-i7 Laptop CPU with 16GB of RAM. Both Mr X and detectives are the same variety of bot.

7.2. Performance

Hider	Detectives			
		One-Step	Minimax	MCTS
	One-Step	40%	25%	5%
	Minimax	55%	60%	10%
	MCTS	85%	80%	55%

Fig 6: Win rate of Mr X against the detectives (20 games each). Mr X MCTS used root parallelisation. Detectives use root parallelisation and single tree determinisation.

Future Work / Limitations

8.1. Benchmarking

Most of our benchmarks were run for 20 iterations. It is hard to draw definitive conclusions from so few experiments. More rigorous testing would allow for greater confidence in the findings of this report.

8.2. Location categorisation

In our current implementations of expectimax minimax and MCTS, we assign all Mr X possible with equal probabilities. Location categorisation improves the model by biasing it towards more likely locations of Mr X. This was shown by [1] to have a significant effect.

References

- [1] Nijssen, P., & Winands, M. H. M. (2012). Monte Carlo Tree search for the Hide-and-Seek Game Scotland Yard. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(4), 282–294. <https://doi.org/10.1109/tciaig.2012.2210424>
- [2] Chaslot, G. M. J. -, Winands, M. H. M., & Van Den Herik, H. J. (2008). Parallel Monte-Carlo tree search. *In Lecture notes in computer science* (pp. 60–71). https://doi.org/10.1007/978-3-540-87608-3_6